

Tentu, ini adalah penjelasan mendalam mengenai kode AdaptiveHistogram.cs.

Gambaran Umum

Kode ini mengimplementasikan algoritma **Adaptive Histogram Equalization (AHE)** dengan interpolasi. Berbeda dengan *Histogram Equalization* biasa yang mengubah kontras berdasarkan data *seluruh* gambar, metode ini membagi gambar menjadi kotak-kotak kecil (grid/tile), menghitung perbaikan kontras untuk setiap kotak, lalu menggabungkannya kembali dengan teknik penghalusan (interpolasi) agar tidak terlihat kotak-kotak.

Ini adalah teknik yang sangat ampuh untuk memperbaiki foto yang memiliki area gelap dan terang secara bersamaan.

1. Library dan Namespace

C#

```
using System;  
using System.Drawing;  
using System.Drawing.Imaging;  
using System.Runtime.InteropServices;  
using System.Threading.Tasks;  
  
namespace MiniPhotoshop.Logic.Histogram
```

- **System.Drawing & System.Drawing.Imaging:** Pustaka inti untuk memanipulasi gambar (Bitmap, PixelFormat, LockBits).
- **System.Runtime.InteropServices:** Digunakan untuk Marshal.Copy, yaitu menyalin data dari memori gambar (pointer) ke array byte[] agar pemrosesan jauh lebih cepat daripada GetPixel/SetPixel.
- **System.Threading.Tasks:** Digunakan untuk Parallel.For, yang memungkinkan kode berjalan menggunakan banyak core CPU sekaligus (multithreading) untuk mempercepat proses.

2. Kelas dan Constructor

C#

```

public class AdaptiveHistogram
{
    private int _gridRows;
    private int _gridCols;

    public AdaptiveHistogram(int gridRows = 8, int gridCols = 8)
    {
        _gridRows = gridRows;
        _gridCols = gridCols;
    }
}

```

- **_gridRows, _gridCols:** Variabel untuk menyimpan berapa banyak kotak pembagian gambar.
- **Constructor:** Secara default membagi gambar menjadi grid **8x8**. Ini adalah standar umum dalam pengolahan citra. Semakin banyak grid, semakin detail adaptasi kontras lokalnya, namun komputasi semakin berat.

3. Metode Utama: Apply

Metode ini adalah otak dari proses. Kita akan membedahnya per tahap.

A. Persiapan Memori (LockBits)

C#

```

public Bitmap Apply(Bitmap source)
{
    int w = source.Width;
    int h = source.Height;

    Bitmap dst = (Bitmap)source.Clone();
    BitmapData dstData = dst.LockBits(new Rectangle(0, 0, w, h),
        ImageLockMode.ReadWrite, PixelFormat.Format32bppArgb);

    int bytes = Math.Abs(dstData.Stride) * h;
    byte[] buffer = new byte[bytes];
    Marshal.Copy(dstData.Scan0, buffer, 0, bytes);
    int stride = dstData.Stride;
}

```

- **source.Clone()**: Kita menduplikasi gambar agar gambar asli tidak rusak.
- **LockBits**: Ini mengunci memori gambar agar kita bisa mengakses byte pixelnya secara langsung. Ini **sangat krusial** untuk performa.
- **Marshal.Copy**: Menyalin seluruh data pixel dari memori komputer ke dalam array byte[] buffer. Sekarang kita bisa mengedit buffer dengan sangat cepat.
- **Stride**: Lebar satu baris data gambar di memori (termasuk padding). Ini penting untuk navigasi array 1 dimensi sebagai gambar 2 dimensi.

B. Langkah 1: Hitung Ukuran Blok

C#

```
// 1. Hitung Ukuran Blok
int tileW = (int)Math.Ceiling((double)w / _gridCols);
int tileH = (int)Math.Ceiling((double)h / _gridRows);
```

- Menghitung lebar (tileW) dan tinggi (tileH) pixel untuk setiap kotak grid. Math.Ceiling digunakan agar jika pembagian tidak bulat, kita membulatkan ke atas agar seluruh gambar tertutup grid.

C. Langkah 2: Hitung CDF (Mapping) per Tile

C#

```
// 2. Hitung CDF untuk SETIAP TILE (Disimpan di Array 3D)
// maps[row, col, intensity]
int[,] mapsR = new int[_gridRows, _gridCols, 256];
int[,] mapsG = new int[_gridRows, _gridCols, 256];
int[,] mapsB = new int[_gridRows, _gridCols, 256];
```

- Kita membuat **Array 3 Dimensi** ([BarisGrid, KolomGrid, NilaiWarna]).
- Array ini akan menyimpan "Rumus Perbaikan Warna" untuk setiap kotak grid secara terpisah.

C#

```
// Kita hitung dulu semua histogram grid-nya
Parallel.For(0, _gridRows, r =>
{
    for (int c = 0; c < _gridCols; c++)
    {
        // ... (Kode menentukan koordinat startX, endX tile saat ini) ...

        // ... (Looping pixel dalam tile untuk menghitung histogram histR, histG, histB) ...

        // Simpan CDF ke array global
        int[] cdfR = CalculateCDF(histR, count);
        // ... (Simpan ke mapsR[r, c, i]) ...
    }
});
```

- **Parallel.For:** Kita menghitung histogram setiap kotak secara bersamaan (paralel) agar cepat.
- Di dalam loop ini, kode menghitung frekuensi kemunculan warna (Histogram) hanya untuk area kotak tersebut.
- **CalculateCDF:** Mengubah data histogram mentah menjadi kurva pemetaan (Cumulative Distribution Function). Inilah inti dari "Equalization".
- Hasilnya disimpan ke mapsR, mapsG, mapsB. Sekarang setiap kotak punya aturan kontrasnya sendiri.

D. Langkah 3: Interpolasi Bilinear (Bagian Paling Rumit)

Jika kita langsung menerapkan maps per kotak, gambar akan terlihat "pecah" seperti papan catur. Interpolasi digunakan untuk menghaluskan perpindahan antar kotak.

C#

```
// 3. PROSES INTERPOLASI (Menghaluskan batas)
Parallel.For(0, h, y =>
{
    for (int x = 0; x < w; x++)
    {
        // Hitung posisi relatif terhadap grid
```

```
float gridY = (float)(y - tileH / 2.0) / tileH;  
float gridX = (float)(x - tileW / 2.0) / tileW;
```

- Kita loop **setiap pixel** gambar.
- **gridY, gridX**: Kita mencari tahu posisi pixel ini relatif terhadap "pusat" grid. Karena interpolasi dihitung berdasarkan jarak ke pusat grid terdekat.

C#

```
// Cari index tile kiri-atas (r1, c1)  
int r1 = (int)Math.Floor(gridY);  
int c1 = (int)Math.Floor(gridX);
```

```
// ... (Logika mencari r2, c2 dan clamping agar tidak error array index out of bound) ...
```

- Kode ini menentukan **4 kotak tetangga** terdekat dari pixel tersebut (Kiri-Atas, Kanan-Atas, Kiri-Bawah, Kanan-Bawah).

C#

```
// Hitung fraksi (jarak) untuk interpolasi (0.0 s.d 1.0)  
float alpha = gridY - (float)Math.Floor(gridY); // Jarak vertikal  
float beta = gridX - (float)Math.Floor(gridX); // Jarak horizontal
```

- **alpha & beta**: Nilai persentase (0.0 sampai 1.0).
 - Jika beta mendekati 0, berarti pixel lebih dekat ke kotak kiri.
 - Jika beta mendekati 1, pixel lebih dekat ke kotak kanan.

C#

```
// Interpolasi B, G, R  
buffer[k] = (byte)BilinearInterpolate(buffer[k], mapsB, r1, r2, c1, c2, alpha, beta);  
// ... (Lakukan juga untuk G dan R) ...
```

```
}  
});
```

- Fungsi ini menggabungkan "saran" kontras dari 4 kotak tetangga tadi berdasarkan jaraknya.

E. Finalisasi

C#

```
Marshal.Copy(buffer, 0, dstData.Scan0, bytes);  
dst.UnlockBits(dstData);  
  
return dst;  
}
```

- Menyalin kembali data buffer yang sudah dimodifikasi ke memori gambar asli.
- Membuka kunci (UnlockBits) dan mengembalikan gambar hasil.

4. Metode Helper: BilinearInterpolate

C#

```
private int BilinearInterpolate(int val, int[,] maps, int r1, int r2, int c1, int c2, float alpha, float beta)  
{  
    // Ambil nilai mapping dari 4 titik sudut  
    int tl = maps[r1, c1, val]; // Top-Left  
    int tr = maps[r1, c2, val]; // Top-Right  
    int bl = maps[r2, c1, val]; // Bottom-Left  
    int br = maps[r2, c2, val]; // Bottom-Right  
  
    // Interpolasi Horizontal  
    float t = (1 - beta) * tl + beta * tr; // Atas  
    float b = (1 - beta) * bl + beta * br; // Bawah  
  
    // Interpolasi Vertikal  
    return (int)((1 - alpha) * t + alpha * b);  
}
```

```
}
```

- Metode ini melakukan matematika pencampuran warna.
- Pertama, ia mengambil nilai target warna (val) dari tabel pemetaan (maps) milik 4 grid tetangga (tl, tr, bl, br).
- Lalu ia menghitung rata-rata tertimbang (weighted average) berdasarkan alpha (vertikal) dan beta (horizontal). Hasilnya adalah transisi kontras yang sangat halus.

5. Metode Helper: CalculateCDF

C#

```
private int[] CalculateCDF(int[] histogram, int totalPixels)
{
    // ... (Logika standar Histogram Equalization) ...
    sum += histogram[i];
    int val = (int)(sum * scale);
    // ...
}
```

- Ini adalah rumus standar matematika statistik.
- Ia menjumlahkan frekuensi secara kumulatif (sum) lalu menormalisasinya ke rentang 0-255.
- Inilah yang membuat distribusi warna menjadi rata (equalized), meningkatkan detail pada area yang sebelumnya kurang kontras.